**PROJETO DE QUALIDADE E TESTE DE SOFTWARE (DEVOPS) — GRUPO TECHCARE**

**Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas**

**INTEGRANTES DO PROJETO e RA'S**



| | | |
|---|---|---|
| **Victor Bancatelli Lucena Lopes** | **-** | **25027658** |
| **Nelson dos Reis Gomes Souza** | **-** | **25027592** |
| **Nicolas Hayato Nitta** | **-** | **25027686** |
| **Karine A Cardoso Alves** | **-** | **25027812** |



São Paulo

2026

## Sumário

**1. Qualidade de Software 6**

**1.1. Modelo que qualidade de software (Diagrama/Design) 6**

**1.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo) 6**

**1.3. Identificação de atributos de qualidade da norma 25010. 6**

**1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento. 6**

**1.5. REFERÊNCIAS BIBLIOGRÁFICAS 6**

## 1. Qualidade de Software

Quando se trata de um sistema que armazena e gerencia dados clínicos de pacientes — prontuários, históricos de dor, planos de exercício individualizados — a qualidade deixa de ser um diferencial e passa a ser uma obrigação. O sistema Maya Yoshiko Yamamoto RPG foi concebido para digitalizar o fluxo de trabalho de uma fisioterapeuta especializada em Reeducação Postural Global, e qualquer falha nesse contexto pode comprometer tanto o acompanhamento terapêutico quanto a privacidade de informações sensíveis de saúde.

Por isso, a qualidade é tratada neste projeto não como uma etapa final de verificação, mas como uma propriedade que precisa ser construída desde o levantamento de requisitos, atravessar todas as decisões de arquitetura e se materializar em critérios concretos de aceitação. O referencial adotado para estruturar essa visão é a norma ISO/IEC 25010.

**Referencial Normativo: ISO/IEC 25010**

A norma ISO/IEC 25010 pertence à família SQuaRE (Systems and Software Quality Requirements and Evaluation) e define um modelo de qualidade de produto organizado em oito características principais — cada uma desdobrada em subcaracterísticas mensuráveis. Ela permite que o grupo saia do campo subjetivo do "sistema bom" e trabalhe com perguntas objetivas: o sistema responde em tempo adequado? Resiste a acessos não autorizados? Pode ser mantido sem introduzir novos erros?

O modelo é avaliado em duas dimensões complementares:

Qualidade funcional: trata de o sistema fazer o que foi especificado — os exercícios aparecem corretamente para cada paciente, o check-in é registrado com a data e escala de dor certas, o prontuário reflete o histórico real.

Qualidade estrutural: trata de como o sistema se comporta além das funcionalidades visíveis — quão rápido responde, quão seguro é ao guardar dados de saúde, quão fácil é para um novo membro da equipe entender e modificar o código sem quebrar o que já funciona.

Como a Qualidade Será Medida e Garantida?

Para que qualidade não fique no campo das intenções, o grupo definiu práticas concretas de medição e controle ao longo de todo o ciclo de desenvolvimento:

**Medição contínua:** métricas como cobertura de testes unitários (meta mínima de 70% nas classes de lógica de negócio), tempo de resposta das principais rotas da API (abaixo de 2 segundos em condições normais) e número de bugs abertos por sprint serão monitoradas durante o desenvolvimento.

**Garantia preventiva:** revisão de código obrigatória antes de qualquer merge, análise estática com ESLint e Prettier, e definição de critérios de aceite no backlog antes do início de cada tarefa — evitando retrabalho ao final.

**Controle por testes:** ciclos estruturados de testes unitários, de integração e com usuários garantem que os requisitos funcionais e não funcionais sejam verificados de forma independente, e não apenas ao final da entrega.

## 1.1. Modelo que qualidade de software (Diagrama/Design)

O diagrama abaixo representa o modelo de processo de desenvolvimento orientado à qualidade adotado pelo grupo. Ele foi elaborado para refletir a realidade iterativa do projeto: nenhuma fase avança sem uma decisão de aprovação, e qualquer não conformidade identificada — seja na prototipação, nos testes ou na validação com o usuário — dispara um ciclo de correção antes da continuidade.

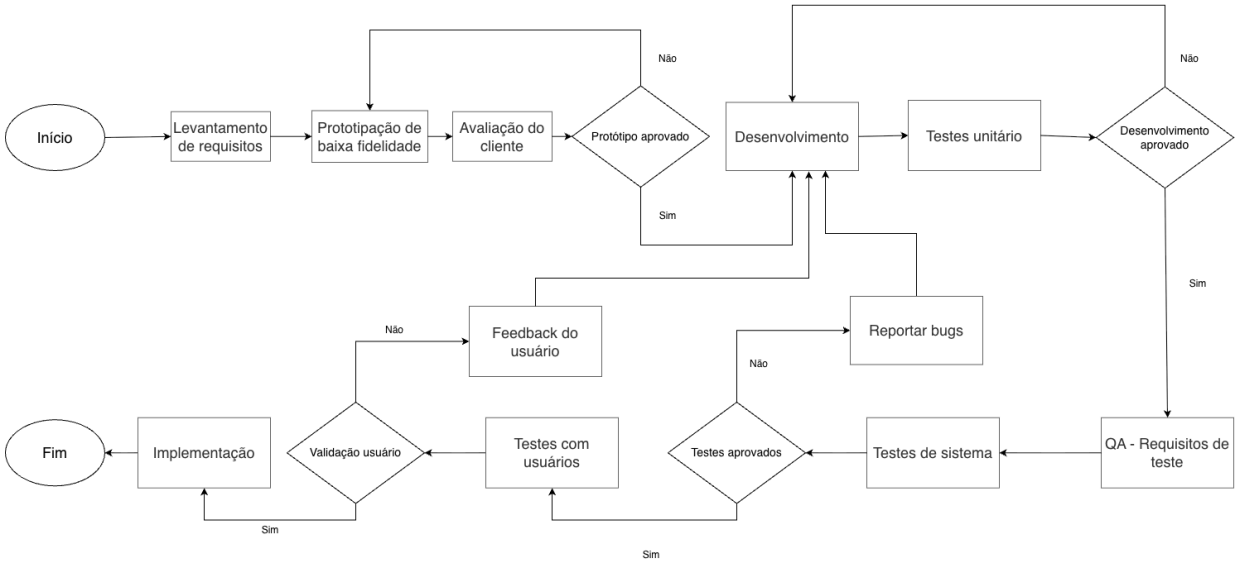


Figura 1 – Diagrama do processo de desenvolvimento orientado à qualidade para o sistema Maya Yoshiko Yamamoto RPG.

O fluxo começa com o levantamento de requisitos junto à fisioterapeuta Maya, passa por uma prototipação de baixa fidelidade submetida à avaliação da cliente e só entra em desenvolvimento após aprovação formal do protótipo. Durante o desenvolvimento, testes unitários são executados continuamente; se o código não for aprovado nessa camada, retorna ao ciclo de desenvolvimento antes de avançar.

Aprovado o desenvolvimento, uma etapa dedicada de QA define os requisitos de teste específicos que orientam a execução dos testes de sistema. A partir daí, dois caminhos de validação se somam: testes técnicos de sistema (que podem retornar à correção de bugs se reprovados) e testes com usuários reais, cuja validação pode gerar feedback que retorna ao desenvolvimento para ajustes. Apenas após a validação do usuário o ciclo avança para a implementação final.

Esse modelo garante que atributos como Adequação Funcional, Usabilidade e Confiabilidade sejam verificados em camadas distintas — e não todos de uma vez ao final, quando o custo de correção é muito maior.

## 1.2. Processo (plano) de gerenciamento de qualidade de software

O plano de gerenciamento de qualidade descreve como o grupo vai planejar, monitorar e controlar a qualidade do software ao longo das 13 semanas de desenvolvimento. A lógica central é simples: qualidade não acontece no final — ela precisa estar embutida em cada decisão de código, em cada revisão e em cada conversa com a cliente.

**Planejamento:** qualidade antes de escrever código

Antes de qualquer linha de implementação, o grupo alinha o que "pronto" significa para cada funcionalidade. Isso inclui definir os critérios de aceite no backlog (ex: "o paciente consegue registrar um exercício em no máximo três toques, sem mensagens de erro genéricas"), mapear os riscos técnicos de cada entrega e garantir que os requisitos não funcionais estejam tão bem especificados quanto os funcionais.

A arquitetura em camadas — Controller, Service e Repository no backend; componentes reutilizáveis no React Native — é decidida nesta fase justamente para garantir que futuras mudanças possam ser feitas sem efeito cascata indesejado.

Controle durante o desenvolvimento

As práticas de controle de qualidade estão integradas ao fluxo diário de trabalho do grupo:

- **Versionamento com Git Flow:** branches separadas por funcionalidade, com merge apenas após revisão. O histórico de commits segue o padrão Conventional Commits, facilitando rastreabilidade e geração de changelogs.
- **Code review obrigatório:** nenhum código entra na branch principal sem revisão de pelo menos um outro membro. O foco é em lógica, clareza e conformidade com os padrões definidos — não apenas em "funciona".
- **Linting e formatação automática:** ESLint e Prettier configurados no projeto garantem consistência de estilo sem depender de revisão manual para aspectos triviais.
- **Testes escritos junto com o código:** funções de negócio são desenvolvidas com seus testes unitários correspondentes na mesma sprint, evitando o acúmulo de dívida técnica em testes.

**Responsabilidades e critérios de aceite**

A qualidade técnica do código é responsabilidade de quem o escreve — mas a aprovação depende da revisão do par. A adequação funcional é validada pelo grupo completo nos testes de sistema e, no teste final, pela própria cliente. Uma tarefa só transita para "concluída" quando passa pelos testes automatizados e pela revisão de código; funcionalidades que afetam o paciente passam adicionalmente por validação de usabilidade.

**Tratamento de falhas e melhoria contínua**

Bugs encontrados em qualquer fase são registrados como issues no GitHub com nível de severidade (crítico, moderado ou baixo) e só avançam no fluxo quando resolvidos. Nenhuma funcionalidade com bug crítico aberto entra na entrega final. Ao fim de cada ciclo de entrega, o grupo realiza uma retrospectiva breve para identificar o que gerou mais retrabalho e ajustar o processo para a fase seguinte.

## 1.3. Identificação de atributos de qualidade da norma 25010.

Das oito características definidas pela ISO/IEC 25010, cinco foram identificadas como centrais para o sistema Maya Yoshiko Yamamoto RPG, considerando o perfil dos usuários, a natureza dos dados tratados e as limitações de escopo do projeto. A seleção não descarta as demais características — mas reconhece que foco e profundidade importam mais do que cobrir superficialmente todas as dimensões.

| Característica | Subcaracterística | Critério de Aceitação | Como é verificado |
|---|---|---|---|
| Adequação Funcional | Completude e Correção | Todas as funcionalidades do escopo operam conforme especificado, sem desvios de comportamento | Testes de sistema e critérios de aceite do backlog |
| Segurança | Confidencialidade e Integridade | Dados de saúde inacessíveis sem autenticação válida; senhas não armazenadas em texto puro | Testes de autenticação, auditoria de código, verificação de endpoints |
| Usabilidade | Operabilidade e Apreensibilidade | Paciente conclui registro de exercício em até 3 interações; taxa de completude de tarefas acima de 85% em testes com novos usuários | Testes com usuários reais, avaliação de protótipo com a cliente |
| Confiabilidade | Maturidade e Disponibilidade | Sistema sem erros nos fluxos principais; acesso parcial disponível offline via cache local | Testes de regressão, validação de comportamento sem conexão |
| Manutenibilidade | Modularidade e Testabilidade | Cobertura mínima de 70% em testes unitários nas classes de lógica de negócio; alterações em um módulo não afetam os demais | Relatório de cobertura, code review, análise estática |

Tabela 1 – Atributos de qualidade prioritários e seus critérios de verificação.

**Adequação Funcional**

É o ponto de partida da qualidade: o sistema precisa fazer o que foi prometido. Para o contexto clínico da Maya, isso significa que o plano de exercícios deve ser exibido corretamente para cada paciente, o check-in precisa registrar data e escala de dor com precisão e o prontuário deve refletir o histórico real sem inconsistências. Funcionalidades incompletas ou com comportamento incorreto comprometem diretamente o acompanhamento terapêutico.

**Segurança**

O sistema lida com dados sensíveis de saúde, categoria especialmente protegida pela Lei Geral de Proteção de Dados (Lei nº 13.709/2018). Isso eleva a segurança ao mesmo nível de prioridade das funcionalidades principais. O acesso ao sistema é protegido por autenticação com JWT, as senhas são armazenadas com hash bcrypt, e cada perfil de usuário — paciente, profissional e administrador — possui permissões específicas que impedem acessos indevidos a dados de terceiros.

**Usabilidade**

O aplicativo mobile será utilizado por pacientes em casa, sem suporte técnico disponível. Isso significa que a interface precisa ser autoexplicativa: se um paciente com pouca familiaridade com tecnologia não conseguir registrar um exercício sozinho, a funcionalidade falha — independentemente de estar tecnicamente correta. A usabilidade será avaliada com protótipos validados pela cliente e com testes de uso durante o desenvolvimento.

**Confiabilidade**

Pacientes realizam exercícios em ambientes variados, nem sempre com boa conexão à internet. O sistema precisa ser confiável mesmo em condições adversas — e isso se traduz em cache local via SQLite no app mobile, que garante acesso ao plano de exercícios offline. No backend, tratamento adequado de erros impede que falhas pontuais comprometam o histórico clínico já registrado.

**Manutenibilidade**

O projeto tem prazo de 13 semanas e será desenvolvido por um grupo com diferentes ritmos e especialidades. Código difícil de entender gera retrabalho, bugs silenciosos e dificuldade de integração. A arquitetura em camadas e a cobertura mínima de testes unitários não são apenas boas práticas — são a garantia de que cada membro consegue modificar uma parte do sistema sem comprometer o restante.

## 1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

A norma ISO/IEC 25010 funciona como uma lente que o grupo aplica sobre cada fase do projeto para identificar o que poderia passar despercebido em um desenvolvimento focado apenas nas funcionalidades visíveis. A seguir, descrevemos como ela se traduz em decisões reais em cada etapa.

**Na fase de requisitos: tornando o invisível visível**

Requisitos funcionais são naturais de levantar a cliente diz o que quer, o grupo escreve. O desafio está nos requisitos não funcionais, que raramente aparecem em uma conversa inicial. A norma serviu como checklist para provocar perguntas que de outra forma não seriam feitas: o sistema precisa funcionar sem internet? (Confiabilidade – Disponibilidade.) Quanto tempo o paciente pode esperar por uma tela? (Eficiência de Desempenho.) Quem pode ver o prontuário de qual paciente? (Segurança – Confidencialidade.) Cada resposta virou um requisito rastreável, com critério de aceite definido.

**Na arquitetura: qualidade como decisão de design**

As escolhas de tecnologia e estrutura de código foram feitas com os atributos da norma em mente. A separação em camadas Controller, Service e Repository no backend atende diretamente à Manutenibilidade — qualquer alteração de regra de negócio fica isolada na camada de serviço, sem tocar nos endpoints nem no banco de dados. O armazenamento local em SQLite no app mobile foi escolhido para atender à Confiabilidade em cenários sem conexão. A autenticação com JWT e controle de acesso por perfil endereça a Segurança desde a fundação, não como um acréscimo posterior.

**No desenvolvimento: qualidade verificada em tempo real**

A norma guia a construção dos testes. Testes unitários focam em Correção Funcional e Manutenibilidade — verificam se a lógica de negócio produz o resultado esperado de forma isolada, e sua existência garante que mudanças futuras não quebrem silenciosamente o que já funciona. Testes de integração verificam Compatibilidade e Correção na comunicação entre API, banco de dados e autenticação. O code review com foco em análise estática atende à Segurança em cada commit, e não apenas em auditorias pontuais.

**Na validação com o usuário: fechando o ciclo com Usabilidade**

Nenhum atributo de qualidade é mais difícil de medir internamente do que a Usabilidade — o grupo pode achar a interface intuitiva e o usuário real pode travar no primeiro passo. Por isso, o fluxo de processo (Figura 1) prevê explicitamente ciclos de teste com usuários antes da implementação final. O feedback coletado nesses testes retorna ao desenvolvimento como ajustes priorizados, garantindo que o produto entregue seja validado por quem realmente o utiliza.

**Métricas que orientam as decisões**

Para que a aplicação da norma seja objetiva e não apenas declarativa, o grupo estabeleceu metas mensuráveis derivadas das subcaracterísticas da 25010:

1. **Tempo de resposta da API:** abaixo de 2 segundos para as rotas principais em condições normais de uso — endereça Eficiência de Desempenho.
2. **Cobertura de testes unitários:** mínimo de 70% nas classes de lógica de negócio — endereça Manutenibilidade e Testabilidade.
3. **Taxa de completude de tarefas:** acima de 85% em testes com usuários que nunca usaram o sistema — endereça Usabilidade e Apreensibilidade.
4. **Bugs críticos em produção:** zero funcionalidades com bugs críticos abertos avançam para a entrega — endereça Confiabilidade e Maturidade.

Essas metas tornam cada aprovação no fluxo de desenvolvimento (representada pelos losangos do diagrama) algo concreto e verificável — não uma avaliação subjetiva de "está bom o suficiente".

## 1.5. REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR ISO/IEC 25010:2023: Engenharia de sistemas e de software — Requisitos e avaliação de qualidade de sistemas e de software (SQuaRE) — Modelos de qualidade de sistema e de produto de software. Rio de Janeiro: ABNT, 2023.

ISO/IEC. ISO/IEC 25010:2011: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. Geneva: International Organization for Standardization, 2011.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

WIEGERS, Karl; BEATTY, Joy. Software Requirements. 3. ed. Redmond: Microsoft Press, 2013.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, 2018.

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.